

Overflow error when simulating large circuits

<https://github.com/tensorflow/quantum/issues/442>

ROW 63

Overflow error when simulating large circuits #442

New issue

Closed



AlkaidCheng opened on Nov 27, 2020 · edited by MichaelBroughton

Edits ...

Platform: GCP N2 CPU, 48 vCPUs, 384 GB memory

Package Versions:

tensorflow == 2.3.1

tensorflow_quantum == 0.4.0

cirq == 0.9.1

Issue:

When using tensorflow quantum state layer for final state simulation, if the resulting size of the final state arrays exceed 2^{31} there will be an overflow error.

The error message is:

```
2020-11-27 14:24:49.630097: F tensorflow/core/grappler/optimizers/constant_folding.cc:1683] Non-OK-status:
TensorShapeUtils::MakeShape(shp, &new_dims) status: Invalid argument: Dimension -2147483648 must be >= -1
```

```
import numpy as np
import sympy
import cirq
from cirq import Circuit, GridQubit
import tensorflow_quantum as tfq

n_qubit = 20
n_circuits = 2**11

# construct parameterised circuit
cq = Circuit()
thetas = sympy.symbols('θ', n_qubit)
qubits = GridQubit.rect(1, n_qubit)
cq.append([cirq.rz(theta)(qubit) for theta, qubit in zip(thetas, qubits)])

# simulate final states
x = np.random.rand(n_circuits, n_qubit)
state_layer = tfq.layers.State()
symbol_names = [theta.name for theta in thetas]
states = state_layer(cq, symbol_names=symbol_names, symbol_values=x)
```

The above code runs when $n_circuits = 2^{11} - 1$

For simulating the final states of 2^{11} circuits of 20 qubits, the expected size of the final states array will be $(2^{20}) \cdot (2^{11}) = 2^{31}$. I believe it is due to an overflow error for a signed integer that holds the final output dimension.



MichaelBroughton on Nov 29, 2020 · edited by MichaelBroughton

Edits Collaborator ...

Hi @AlkaidCheng, thanks for raising this issue.

You are right! This is caused by pip TensorFlow's underlying C++ Tensor classes using 32 bit integers for their size. You can see here in this snippet too:

```
>>> a = tf.constant(0, shape=(2**33))
>>> a
<tf.Tensor: shape=(0,), dtype=int32, numpy=array([], dtype=int32)>
```

This isn't something that we intend to fix and I would suggest that you try and decrease your batch size (`n_circuits`) and use multiple calls to the `state_layer` to keep intermediate sizes down. More concretely you could split your `x` into strides of size `k` and then make multiple calls for each subsection of `x`.

For example if I wanted to compute a kernel matrix between state vectors, but I couldn't store all the state vectors in memory at once, I could do something like:

Assignees

MichaelBroughton

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



This isn't something that we intend to fix and I would suggest that you try and decrease your batch size (`n_circuits`) and use multiple calls to the `state_layer` to keep intermediate sizes down. More concretely you could split your `x` into strides of size `k` and then make multiple calls for each subsection of `x` .

For example if I wanted to compute a kernel matrix between state vectors, but I couldn't store all the state vectors in memory at once, I could do something like:

```
# Here data_batch is a list or tensor of `cirq.Circuits`.
def compute_kernel_from_states(data_batch):
    """Computes  $d[i][j] = |\langle \psi_i | \psi_j \rangle|^2$  """
    stride = 50 # must be multiple of len(data_batch)
    batch_dim = tf.gather(tf.shape(data_batch), 0)
    empty_symbols = tf.convert_to_tensor([], dtype=tf.dtypes.string)
    empty_values = tf.tile(tf.convert_to_tensor([[]]), [stride, 1])
    ref_others = tf.reshape(tf.tile(data_batch, [stride]), [stride, batch_dim])

    res = np.empty((batch_dim, batch_dim))
    for i in range(0, batch_dim, stride):
        print('Processing state overlaps: ', i, '/', batch_dim.numpy())
        interim = tfq.math.inner_product(data_batch[i:i+stride], empty_symbols, empty_values, ref_others)
        res[i:i+stride] = (tf.math.abs(interim) ** 2).numpy()

    return res
```

This strided approach to keeping intermediate memory sizes down becomes more and more important as qubit numbers go up, suppose you wanted to compute this kernel for 28 qubit circuits with 2^{11} states that'd be over 4TB of RAM without striding, still far too big for a TensorFlow Tensor :). We actually used this and some other tricks in [here](#) for the quantum kernel methods.

It's also worth mentioning that all TFQ ops that do not rely on outputting large `tf.Tensors` can still very readily go over 20 qubits. If you changed this op to a sample op or expectation op you wouldn't need to worry about these sorts of things and could go well into 30+ qubits since the number of outputs you need to store in the final `tf.Tensor` that the op produces is much smaller.

Going to close this for now since this isn't an issue we plan on fixing.

Hope this information can help.

Michael



MichaelBroughton self-assigned this on Nov 29, 2020



MichaelBroughton closed this as **completed** on Nov 29, 2020



jaeyoo added a commit that references this issue on Mar 30, 2023

Merge pull request [tensorflow#442](#) from quantumlib/qsim-gpu-gcp ...

Verified 2cda0e9